

UGANDA CHRISTIAN UNIVERSITY
CSC2105 OBJECT-ORIENTED PROGRAMMING USING PYTHON

GROUP 2

**HOSPITAL OUTPATIENT MANAGEMENT
SYSTEM**

Short Object-Oriented Design Report

This report explains the identified classes, encapsulation decisions, object relationships, inheritance hierarchy, polymorphism and abstraction implemented in the project.

1. Project Overview

The Hospital Outpatient Management System is a menu-driven Python application for registering patients and health workers, creating and managing consultations, searching records, viewing consultation histories, calculating consultation charges and displaying consultation summaries. The design is centred on collaborating objects rather than placing all program logic in the menu.

2. Identified Classes and Responsibilities

The main classes were identified from the people, worker categories, consultation transactions and coordination responsibilities in the outpatient scenario.

Class	Main Responsibility
Person (abstract)	Stores common personal identity, names and phone information and defines the common <code>get_role()</code> contract.
Patient	Represents one registered outpatient and adds date of birth, address and a computed age.
HealthWorker (abstract)	Stores common worker information and defines the <code>calculate_charge()</code> and <code>provide_service()</code> contracts.
Doctor	Represents a doctor, adds specialty and implements doctor-specific role, service and charge behaviour.
Nurse	Represents a nurse and implements nurse-specific role, service and charge behaviour.
ClinicalOfficer	Represents a clinical officer and implements clinical-officer-specific role, service and charge behaviour.
Consultation	Represents one outpatient interaction between exactly one Patient and one HealthWorker and controls complaint, service, charge and consultation status.
HospitalSystem	Coordinates registered patients, health workers and consultations; generates IDs; performs searches, histories, consultation transactions and summaries.

The classes are separated by responsibility so that patient data, worker-specific behaviour, consultation state and system-wide coordination are not mixed together.

3. Encapsulation Decisions

Private state. Important object state is stored using private attributes. Examples include `Person.__id`, `Person.__first_name`, `Patient.__date_of_birth`, `HealthWorker.__base_fee` and `Consultation.__status`. This prevents unrestricted external modification.

Protected/internal state. The project demonstrates Python's protected-by-convention style through `Person._phone` and `HealthWorker._department`. These values are intended for internal or subclass use rather than direct external modification.

Properties and setters. Public properties provide controlled access to encapsulated data. Validated setters are used for values that may legitimately change, such as names, phone, address, department, base fee and doctor specialty. Read-only properties are used where direct changes would be unsafe, such as IDs and consultation charge.

Controlled consultation state. Consultation.status has no public setter. The status changes only through start(), complete() and cancel(). This preserves the valid lifecycle: SCHEDULED can become CURRENT or CANCELLED, while CURRENT can become COMPLETED or CANCELLED. Completed and cancelled consultations cannot be reopened.

Validation. The application rejects blank required text, future dates of birth, invalid dates, non-positive base fees, missing doctor specialties, unknown IDs and invalid consultation state changes. HospitalSystem also prevents a patient or health worker from having two CURRENT consultations at the same time.

4. Object Relationships

Association: Patient - Consultation. Each Consultation stores a reference to exactly one Patient. One Patient may be linked to zero or many Consultation objects. The multiplicity is Patient 1 to Consultation 0..*.

Association: HealthWorker - Consultation. Each Consultation stores a reference to exactly one HealthWorker. One HealthWorker may handle zero or many consultations. The multiplicity is HealthWorker 1 to Consultation 0..*.

Aggregation: HospitalSystem. HospitalSystem manages collections of Patient, HealthWorker and Consultation objects using dictionaries. The UML therefore models one HospitalSystem aggregating zero or many objects of each of these types.

Object collaboration. When HospitalSystem creates a consultation, it first finds the existing Patient and HealthWorker objects and passes those objects into Consultation. Consultation then calls the worker's provide_service() and calculate_charge() methods. This is direct collaboration between domain objects rather than menu-based procedural logic.

5. Inheritance Hierarchy

The hierarchy is based on genuine 'is-a' relationships. Patient is a Person. HealthWorker is also a Person. Doctor, Nurse and ClinicalOfficer are specialised HealthWorker subclasses.

```
Person (abstract)
|-- Patient
`-- HealthWorker (abstract)
    |-- Doctor
    |-- Nurse
    `-- ClinicalOfficer
```

The subclasses reuse inherited constructor logic through super(). Patient reuses Person's identity, name and phone handling, while Doctor reuses HealthWorker's common worker data and behaviour. This reduces duplicated code and keeps shared responsibilities in the appropriate superclass.

6. Abstraction

Person abstraction. Person inherits from ABC and declares `get_role()` as an abstract method. The class defines what all people in the system have in common but leaves the exact role to concrete subclasses.

HealthWorker abstraction. HealthWorker remains abstract and declares `calculate_charge()` and `provide_service()` as abstract methods. This establishes a common behavioural contract: every concrete health worker must know which service it provides and how its consultation charge is calculated.

The abstract classes therefore define common interfaces without forcing one identical implementation on every subclass.

7. Method Overriding and Polymorphism

Doctor, Nurse and ClinicalOfficer override `get_role()`, `provide_service()` and `calculate_charge()`. The methods have the same names and are called through the common HealthWorker interface, but the result depends on the actual subclass object.

Subclass	<code>get_role()</code>	<code>provide_service()</code>	Charge Rule
Doctor	Doctor	Medical Consultation	base_fee x 1.50
Nurse	Nurse	Nursing Assessment	base_fee x 1.00
ClinicalOfficer	Clinical Officer	Clinical Consultation	base_fee x 1.20

Consultation does not contain separate Doctor, Nurse and ClinicalOfficer charge formulas. It simply calls `health_worker.calculate_charge()` and `health_worker.provide_service()`. Python then executes the correct overridden subclass method. This is the main polymorphic behaviour in the project.

8. Conclusion

The final design uses clear class responsibilities, controlled object state, meaningful associations and aggregation, a two-level inheritance hierarchy, abstract behavioural contracts and polymorphic worker behaviour. Patient, HealthWorker, Consultation and HospitalSystem objects collaborate to implement the outpatient workflow while keeping the user-interface code separate from the main business rules. The implementation therefore demonstrates the required object-oriented concepts from Topics 5, 6 and 7 in one coherent hospital outpatient application.